Bachelor's Thesis Computing Science



Radboud University Nijmegen

Thesis Title

Subtitle if you like

Author:
Floris Reuvers
s1096976

First supervisor/assessor:
Dr. Niels van der Weide

Second assessor:
Dr. Engelbert Hubbers

June 9, 2025

Abstract

Brief outline of research questions, results. (The preferred size of an abstract is one paragraph or one page of text.)

Contents

1 Introduction **3**

2 Lambda Calculus **4**
2.1 Introduction to the $\boldsymbol{\lambda}$-calculus 4
2.2 Well-Typedness 4
2.3 $\boldsymbol{\beta}$-Reduction 6
2.3.1 Free and Bound Variables 6
2.3.2 Substitution 7
2.3.3 $\boldsymbol{\alpha}$-equivalence 8
2.3.4 $\boldsymbol{\beta}$-Reduction 9
2.4 Call-by-name $\boldsymbol{\lambda}$-calculus 9
2.5 Call-by-value $\boldsymbol{\lambda}$-calculus 11

3 Call-by-box Lambda Calculus **13**
3.1 Specification of $\boldsymbol{\lambda_b}$ 13
3.1.1 Intuitionistic logic IS4 13
3.1.2 Terms and Types of $\boldsymbol{\lambda_b}$ 13
3.2 Substitution and Free and Bound variables in λ_b 15

4 Embeddings in $\boldsymbol{\lambda_b}$ **19**
4.1 Girard's Embedding 19
4.2 Embedding of $\boldsymbol{\lambda_n}$ into $\boldsymbol{\lambda_b}$ 19
4.3 Gödel's Embedding 21
4.4 Embedding of $\boldsymbol{\lambda_n}$ into $\boldsymbol{\lambda_b}$ 21

5 Formalisation in Agda **24**
5.1 Intro to Agda 24
5.1.1 Hole-Based Programming 26
5.2 Challenges in Formalisation 28
5.2.1 De Bruijn Indices 28
5.2.2 Well-typed Terms 29
5.2.3 Substitution 32
5.2.4 Weakening and Renaming 34
5.2.5 Helper Lemmas 35

5.3 Overview of the Formalisation 38

6 Related Work **39**

7 Conclusions **40**

A Appendix **42**

Chapter 1

Introduction

The introduction of your bachelor thesis introduces the research area, the research hypothesis, and the scientific contributions of your work. A good narrative structure is the one suggested by Simon Peyton Jones [**?**]:

- describe the problem / research question
- motivate why this problem must be solved
- demonstrate that a (new) solution is needed
- explain the intuition behind your solution
- motivate why / how your solution solves the problem (this is technical)
- explain how it compares with related work

Close the introduction with a paragraph in which the content of the next chapters is briefly mentioned (one sentence per chapter).

Starting a new paragraph is done by inserting an empty line like this.

Chapter 2

Lambda Calculus

This section provides a detailed and formal description of the λ-calculus. We define a formal grammar of the λ-calculus and give examples of some terms in the λ-calculus. Then we explain β-reduction, after which we define call-by-name evaluation and call-by-value evaluation.

2.1 Introduction to the λ-calculus

The grammar of the lambda calculus is as follows:

Definition 2.1 (λ-calculus)**.** The terms of the λ-calculus are given by:

$$M, N, P, Q \quad ::= \quad x \mid \lambda x.M \mid MN$$

Terms are denoted by M, N, P or Q and can either be of the form x, $\lambda x.M$ or MN:

- x is a variable, which is a symbol that represents an input.
- $\lambda x.M$ is an abstraction. An abstraction is an anonymous function, where x is the parameter and M is the body of the function.
- MN is a function application, where M and N are terms.

The λ-term $\lambda x.x$ is an abstraction. This specific abstraction is called the identity function and has one input parameter, namely x, and returns the input x. The body of the function is also x in this case and the function is often abbreviated as **I**. The λ-term $(\lambda x.x)y$ is a function application. So the identity function is applied to the variable y.

2.2 Well-Typedness

This section discusses the concept of well-typedness in the context of lambda calculus. We first discuss types, typing judgements and typing rules. We then explore how to use these rules to show that a λ-term is well-typed.

Types of λ-calculus are defined in Definition 2.2, and are either X, which is the base type, or a function type, $A \to A'$ with A and A' being types in λ-calculus.

Definition 2.2. Types of λ-calculus

$$A \quad ::= \quad X \mid \ A \to A'$$

A typing judgement is of the form $\Gamma \vdash M : A$, and it means that we can derive that M has type A from the assumptions in context Γ. A context Γ is a list of variable-type pairs, where each variable is associated with a type. A context is written as $\Gamma = x_1 : A_1, \ldots, x_n : A_n$, where each x_i is a variable and each A_i is a type.

The typing rules are defined in Definition 2.3. The Id rule states that if x has type A in the context, we can conclude that x has type A. The $\to$-I rule states that if we have a term M of type A' in the context $\Gamma, x : A$, we can conclude that from context Γ it follows that $\lambda x.M$ has type $A \to A'$. The $\to$-E rule states that if we have a term M of type $A \to A'$ and a term N of type A, we can conclude that the application MN has type A'.

Definition 2.3. Typing rules of λ-calculus

$$\frac{}{\Gamma, x : A \vdash x : A}\ \text{Id}$$

$$\frac{\Gamma, x : A \vdash M : A'}{\Gamma \vdash \lambda x.M : A \to A'}\ \to\text{-I}$$

$$\frac{\Gamma \vdash M : A \to A' \qquad \Gamma \vdash N : A}{\Gamma \vdash MN : A'}\ \to\text{-E}$$

A λ-term is considered to be *well-typed* if we can derive a typing judgement with the typing rules. The λ-terms that are well-typed can be assigned a type. In Example 2.4, we show that λ-term $(\mathbf{I}(\lambda x.\mathbf{I}x))$ is well-typed by giving the full typing derivation.

Example 2.4. Well-typedness of $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{I}\mathbf{I})$

$$\dfrac{\dfrac{\dfrac{}{y : A \to A \vdash y : (A \to A)}\ \text{Id}}{\vdash \lambda y.y : (A \to A) \to (A \to A)}\ \to\text{-I} \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{x : A, z : A \vdash z : A}\ \text{Id}}{x : A \vdash \lambda z.z : A \to A}\ \to\text{-I} \qquad \dfrac{}{x : A \vdash x : A}\ \text{Id}}{x : A \vdash \mathbf{I}x : A \to A}\ \to\text{-E}}{\vdash \lambda x.\mathbf{I}x : A \to A}\ \to\text{-I}}{\vdash \mathbf{I}(\lambda x.\mathbf{I}x) : A \to A}\ \to\text{-E}$$

In contrast to the λ-term $(\mathbf{I}(\lambda x.\mathbf{I}x))$, the lambda term MM cannot be assigned a type. Since MM is an application, we can only apply the $\rightarrow$-E rule, which would give a contradiction. The λ-term M cannot be both a function and the argument to the same function.

2.3 β-Reduction

To include a notion of computation to the λ-calculus, we discuss β-reduction. This reduction relation is the only way to simplify terms in the λ-calculus. So with β-reduction, we can apply an abstraction to an argument. To do this, we need to define substitution, free and bound variables.

2.3.1 Free and Bound Variables

Let us first define free and bound variables. Let P be any term in the λ-calculus, $FV(P)$ be the set that contains all free variables in P and FB be the set that contains all bound variables in P. We define both FV and BV inductively on P by Definition 2.5 and Definition 2.6. Since P can only be a variable, an abstraction or a function application, P can only be of the form x, $\lambda x.M$ and MN. Therefore, we define FV and BV for x, $\lambda x.M$ and MN.

Definition 2.5. FB

$$\begin{aligned} FV(x) &= \{x\} \\ FV(\lambda x.M) &= FV(M)\backslash\{x\} \\ FV(MN) &= FV(M) \cup FV(N) \end{aligned}$$

Let us now explain FV. The variable x in the λ-term x is free. All free variables in an abstraction are those that are not the parameter of the abstraction. For an application MN, all free variables are all free variables of λ-term M and λ-term N.

Definition 2.6. BV

$$\begin{aligned} BV(x) &= \emptyset \\ BV(\lambda x.M) &= BV(M) \cup \{x\} \\ BV(MN) &= BV(M) \cup BV(N) \end{aligned}$$

Let us now explain BV. The variable x in the λ-term x is a free variable, and the set of bound variables is empty. An abstraction $\lambda x.M$ binds the variable x. So the set of bound variables in an abstraction is the set of bound variables in M plus the variable x. For an application MN, the bound variables are all bound variables of both λ-term M and λ-term N.

We give two examples to illustrate the definitions of free and bound variables. We show that λ-term $\lambda x.yx$ has one free variable, y and one bound variable x.

Example 2.7. FV

$$\begin{aligned} FV(\lambda x.yx) &= FV(yx)\backslash\{x\} \\ &= (FV(x) \cup FV(y))\backslash\{x\} \\ &= (\{x\} \cup \{y\})\backslash\{x\} \\ &= \{x, y\}\backslash\{x\} \\ &= \{y\} \end{aligned}$$

Example 2.8. BV

$$\begin{aligned} BV(\lambda x.xy) &= BV(xy) \cup \{x\} \\ &= (BV(x) \cup BV(y)) \cup \{x\} \\ &= (\emptyset \cup \emptyset) \cup \{x\} \\ &= \{x\} \end{aligned}$$

The following example will demonstrate that using two different variables with the same name in a λ-term is quite tricky: $(\lambda x.x)x$. By the definitions of FV and BV, we can reason that $FV((\lambda x.x)x) = \{x\}$ and $BV((\lambda x.x)x) = \{x\}$. While these sets are the same, they do not refer to the same x. This is quite confusing, and we prefer to write $(\lambda y.y)x$ instead, where y is a new "fresh" variable with a different name. We now have $FV((\lambda y.y)x) = \{x\}$ and $BV((\lambda y.y)x) = \{y\}$. These λ-terms are equivalent by the notion of α-equivalence, which states that λ-terms that differ only by the names of bound variables are considered the same. We discuss this in more detail after we define substitution.

2.3.2 Substitution

Keeping the definitions of free and bound variables in mind, we now define substitution. In this thesis, substitution is denoted as $P[x \mapsto Q]$ and is defined inductively on P by Definition 2.9.

Definition 2.9. Substitution

$x[x \mapsto Q] = Q$
$y[x \mapsto Q] = y$ if $(x \neq y)$
$(MN)[x \mapsto Q] = M[x \mapsto Q]N[x \mapsto Q]$
$(\lambda x.M)[x \mapsto Q] = \lambda x.M$
$(\lambda y.M)[x \mapsto Q] = \lambda z.M[y \mapsto z][x \mapsto Q]$ if $(x \neq y)$

with z a variable defined by:

1. If $x \notin FV(N)$ or $y \notin FV(M)$ then $z = y$
2. Otherwise, z can be any variable such that $z \notin FV(N)$ or $z \notin FV(M)$

The variable z always exists. Choosing a fresh variable for z always satisfies condition two. It might not be clear why we need to use the variable z and one might expect the rule in Definition 2.10 is sufficient.

Definition 2.10. Incorrect Substitution Rule

$$(\lambda y.M)[x \mapsto Q] = \lambda z.M[x \mapsto Q] \text{ if } (x \neq y)$$

However, the following example illustrates why the more complicated rule is necessary. Consider the substitution $(\lambda y.xy)[x \mapsto y]$. The free variable of λ-term $\lambda y.xy$ is x and the bound variable is y. If we use the simple rule above, we get that $(\lambda y.xy)[x \mapsto y] = \lambda y.yy$, the meaning of the λ-term changes. However, the free variable we substituted, is not free anymore. The variable y is not a free variable in the term that we substituted y in. Therefore, we change the bound variable y in $\lambda y.xy$ to a new variable z. We can pick z as z is not a free variable in xy. Following the more complex rule, we get:

$$\begin{aligned}(\lambda y.xy)[x \mapsto y] &= \lambda z.(xy)[y \mapsto z][x \mapsto y] \\ &= \lambda z.yz\end{aligned}$$

2.3.3 α-equivalence

Here, we define α-equivalence. The idea is that renaming bound variables does not change the meaning of a λ-term. For example, the λ-terms $\lambda x.x$ and $\lambda y.y$ are equivalent ($\lambda x.x \equiv_\alpha \lambda y.y$), since renaming all bound variables can result in identical λ-terms. The λ-terms $\lambda x.x$ and $\lambda y.y$ represent the same idea. We define the relation $P \equiv_\alpha P'$ inductively on P and P' in Definition 2.11.

Definition 2.11. α-equivalence

$x \equiv_\alpha y$	if $x = y$
$MN \equiv_\alpha M'N'$	if $M \equiv_\alpha M'$ and $N \equiv_\alpha N'$
$\lambda x.M \equiv_\alpha \lambda y.M'$	if $M \equiv_\alpha M'[y \mapsto x]$ with $x = y$ or $x \notin FV(M')$

The last rule states that abstractions $\lambda x.M$ and $\lambda y.M'$ are equivalent if M and M' are equivalent after substituting y for x in M'. However, x should be equal to y or x should not be a free variable in M'.

Example 2.12. Consider the following:

- $\lambda x.zx$ is equivalent to $\lambda y.zy$, since $zy[y \mapsto x] = zx \equiv_\alpha zx$ and x is not a free variable in zy.
- $\lambda z.zx$ is not equivalent to $\lambda y.zy$, since z is a free variable in zy and x is not equal to y.

2.3.4 β-Reduction

Now, we are ready to discuss β-reduction. In the λ-calculus, there is only one way to simplify terms, which is β-reduction. This β-reduction relation is defined in Definition 2.13 with M and N being terms in the λ-calculus.

Definition 2.13. β-reduction

$$(\lambda x.M)N \to M[x \mapsto N] \qquad (\beta)$$

A λ-term of the form $(\lambda x.M)N$ can be reduced by substituting x by N in M. However, without additional rules, we cannot apply β-reduction to subterms. For example, the following reduction would not be possible: $\lambda x.(\lambda y.y)x \to \lambda x.x$. Therefore, in Definition 2.14, we define the following closure rules: μ, ν and ξ.

Definition 2.14. Closure rules for β-reduction

$$\frac{M \to M'}{MN \to M'N}\,(\mu) \qquad \frac{N \to N'}{MN \to MN'}\,(\nu) \qquad \frac{M \to M'}{\lambda x.M \to \lambda x.M'}\,(\xi)$$

Now, we define $\to_\beta$ as β closed under μ, ν and ξ. So with $\to_\beta$ we can use β-reduction on all subterms if the subterm is of the form $(\lambda x.M)N$. A subterm of the form $(\lambda x.M)N$ is called a β-redex. In one reduction, we may need to use multiple rules. The β-redex that is reduced is underlined. For instance, we need to use the μ and ξ rule in the last example of the following reductions:

Example 2.15. Usage of Closure Rules

$$\begin{aligned}
(\underline{\mathbf{II}})(\mathbf{II}) &\to_\beta \mathbf{I}(\mathbf{II}) && \beta \text{ with } \mu \text{ rule}\\
(\mathbf{II})(\underline{\mathbf{II}}) &\to_\beta (\mathbf{II})\mathbf{I} && \beta \text{ with } \nu \text{ rule}\\
\lambda x.\underline{\mathbf{I}x} &\to_\beta \lambda x.x && \beta \text{ with } \xi \text{ rule}\\
(\mathbf{I}(\lambda x.\underline{\mathbf{I}x}))(\mathbf{II}) &\to_\beta (\mathbf{I}(\lambda x.x))(\mathbf{II}) && \beta \text{ with } \mu \text{ and } \xi \text{ rule}
\end{aligned}$$

We see that β-reduction is a non-deterministic relation, which means that there can be multiple ways to reduce a term. However, sometimes we would only like to reduce a term in one way and have a deterministic relation. To achieve this, we can define subrelations that do not allow all closure rules.

2.4 Call-by-name λ-calculus

In this section, we discuss the call-by-name λ-calculus (λ_n). The reduction rule that is used in this calculus is the same as the β reduction rule. However,

we name the rule β_n (Definition 2.16) to make it clear that it is the reduction rule of λ_n. While call-by-name avoids evaluating the argument of a function, arguments are re-evaluated every time they are used in the body of the function. Lazy evaluation stores the evaluation of each argument, so it does not have to re-evaluate the same argument every time it is used. Therefore, an optimised version of call-by-name evaluation is lazy evaluation, which is used in many programming languages. However, for simplicity, we will only focus on call-by-name.

Definition 2.16. β_n-reduction

$$(\lambda x.M)N \to M[x \mapsto N] \qquad (\beta_n)$$

We define the relation $\to_{\beta_n}$ as β_n closed under μ, ν and ξ. The relation $\to_{\beta_n}$ is the call-by-name reduction relation, which allows reduction of λ-terms under all term formers. Weak reduction, $\to_w$, is defined as β_n closed under μ and ν. The only restriction of $\to_w$ is that it cannot reduce under λ's. Weak reduction will be used later to define a deterministic relation that supports cbn and cbv evaluation in a different λ-calculus. We also define the relation $\to_n$ as β_n closed under μ. So the relation $\to_n$ cannot evaluate the argument of a function, since the ν rule is not allowed with $\to_n$. Call-by-name evaluation is thus defined as $\to_n^*$.

Table 2.1 shows the closure rules for the different reduction relations we have defined thus far. In Example 2.17, we give the complete cbn evaluation of λ-term $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{II})$. The β-redex that is reduced is underlined in each step. Note that $\to_n$ is deterministic, so there is always only one way to apply $\to_n$ on a λ-term.

Relation	$\boldsymbol{\beta}$	$\boldsymbol{\mu}$	$\boldsymbol{\nu}$	$\boldsymbol{\xi}$
$\to_{\beta_n}$	β_n	✓	✓	✓
$\to_n$	β_n	✓		
$\to_w$	β_n	✓	✓	

Table 2.1: Closure Rules for Different Reduction Relations

Example 2.17. Cbn evaluation of $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{II})$

$$\begin{aligned}(\underline{\mathbf{I}(\lambda x.\mathbf{I}x)})(\mathbf{II}) &\to_n \underline{(\lambda x.\mathbf{I}x)(\mathbf{II})}\\ &\to_n \underline{\mathbf{I}(\mathbf{II})}\\ &\to_n \underline{\mathbf{II}}\\ &\to_n \mathbf{I}\end{aligned}$$

2.5 Call-by-value λ-calculus

Before we define the call-by-value λ-calculus (λ_v), we first define the notion of values. Values are λ-terms that are fully evaluated. That is, values are λ-terms that cannot be reduced any further and thus do not contain any function applications. We define values Definition 2.18 as all λ-terms that are either a variable or an abstraction. We denote values by V or W.

Definition 2.18. Values in $\boldsymbol{\lambda_v}$

$$V \quad ::= \quad x \mid \ \lambda x.M$$

The main idea of λ_v is that we only reduce a function application if the argument is a value. The β-reduction rule is defined in Definition 2.19.

Definition 2.19. $\boldsymbol{\beta_v}$-reduction

$$(\lambda x.M)V \to M[x \mapsto V] \qquad (\beta_v)$$

We define a new closure rule, $\nu_<$, which is similar to the ν rule, but requires the left λ-term to be a value. In call-by-value evaluation, we first reduce the left λ-term of a function application, to an abstraction, before reducing the right λ-term to a value to apply β_v.

Definition 2.20. $\boldsymbol{\nu_<}$

$$\frac{N \to N'}{VN \to VN'}\,(\nu_<)$$

We define call-by-value reduction, $\to_{\beta_v}$, as β_v closed under μ, ν and ξ which allows reduction of λ-terms under all term formers. The relation $\to_v$ is defined as β_v, closed under μ and $\nu_<$. Table 2.2 shows the closure rules for all different reduction relations we have defined thus far.

Relation	$\boldsymbol{\beta}$	$\boldsymbol{\mu}$	$\boldsymbol{\nu}$	$\boldsymbol{\nu_<}$	$\boldsymbol{\xi}$
$\to_{\beta_n}$	β_n	✓	✓	✓	✓
$\to_n$	β_n	✓			
$\to_w$	β_n	✓	✓		
$\to_{\beta_v}$	β_v	✓	✓	✓	✓
$\to_v$	β_v	✓		✓	

Table 2.2: Closure Rules for Different Reduction Relations

Note that $\to_v$ is deterministic, so there is only one way to apply $\to_v$ on a λ-term. The choice between using μ, $\nu_<$ or no closure rule for $\to_v$ is deterministic. Consider the λ-term MN.

1. If M is an abstraction and N is a value, we can only apply β_v directly.
2. If M is an abstraction and N is an application, we can only apply $\nu_<$ to reduce the left λ-term.
3. If M is an application and N any λ-term, we can only apply μ to reduce the left λ-term.

The closure rule $\nu_<$ forces evaluation from left to right, hence $<$ in the name. First the closure rule μ is used after which $\nu_<$ is used. In Example 2.21, we give the cbv evaluation of λ-term $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{II})$. Again, the β-redex that is reduced is underlined.

Example 2.21. Cbv evaluation of $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{II})$

$$\begin{array}{rcl} (\underline{\mathbf{I}(\lambda x.\mathbf{I}x)})(\mathbf{II}) & \to_v & (\lambda x.\mathbf{I}x)(\underline{\mathbf{II}}) \\ & \to_v & \underline{(\lambda x.\mathbf{I}x)\mathbf{I}} \\ & \to_v & \underline{\mathbf{II}} \\ & \to_v & \mathbf{I} \end{array}$$

Chapter 3

Call-by-box Lambda Calculus

3.1 Specification of λ_b

3.1.1 Intuitionistic logic IS4

Intuitionistic logic IS4 is a non-classical logic that is based on the idea of constructive proof. Unlike classical logic, intuitionistic logic does not allow for the law of excluded middle, which states that for every proposition P, the statement $P \vee \neg P$ assumed. In intuitionistic logic, a proposition is only considered true if it can be constructively proven. That is, a proposition is true if we can explicitly construct a proof, much like writing a program. This is why programming languages and proofs in systems like Agda and Coq are based on intuitionistic logic. Since λ-calculus is the foundation of functional programming, it corresponds with intuitionistic logic via the Curry-Howard correspondence, where programs are equivalent to proofs and types to propositions.

The intuitionistic modal logic IS4 is an extension of intuitionistic logic that introduces *modal operator* $\Box$. The $\lambda_\Box$ calculus is a fragment of IS4 that, besides the usual λ-terms, also uses box (box) and unbox (ε) operators and is strong enough to interpret both call-by-name (cbn) and call-by-value (cbv) lambda calculi [6]. The box is like a little container that delays a computation, making it possible to simulate both cbn and cbv evaluation strategies. In this thesis, we formalize the *boxed lambda calculus* in Agda, denoted as λ_b, which is as a refinement of the simpler $\lambda_\Box$ calculus.

3.1.2 Terms and Types of λ_b

We now define the types of the λ_b calculus. We indicate the types of λ_b with A and B, representing types and boxed types respectively. The type

X denotes the base type. Note that functions types are of the form $B \to A$ thus the input is always a boxed type.

Definition 3.1. Types of λ_b

$$A \quad ::= \quad X \mid \ B \to A \mid \ B \qquad\qquad B \quad ::= \quad \Box A$$

We give some examples of types in λ_b. Note that A can be any type that is not boxed, while B can be any type that is boxed. The type X is the base type.

Example 3.2. Types in λ_b

$$X$$
$$\Box(B \to A)$$
$$\Box X \to X$$
$$\Box A \to \Box B$$
$$\Box\Box\Box X \to \Box X$$

Since variables are always boxed, variables in λ_b are always inside the unboxing operator ε. Thus, we only have $\varepsilon(x)$ in the grammar with x a variable. Abstractions in λ_b, take boxed terms as arguments, which is why we define abstractions as $\lambda x.M$, where x is the parameter of a boxed type and M is a term. The rest of the terms are defined in Definition 3.3 and some example terms are given in Example 3.4.

Definition 3.3. Terms in λ_b

$$M, N, P, Q \quad ::= \quad \varepsilon(x) \mid \ \lambda x.M \mid \ MN \mid \ \mathsf{box}(N)$$

Example 3.4. Terms in λ_b

$$\varepsilon(x)$$
$$\lambda x.\varepsilon(x)$$
$$(\lambda x.\varepsilon(x))\mathsf{box}(\varepsilon(y))$$
$$(\lambda x.\varepsilon(x))\mathsf{box}(\lambda y.\varepsilon(y))$$

The typing rules of λ_b are defined in Definition 2.3. The $\Box$-I rule states that if M is of type A, then $\mathsf{box}(M)$ is of type $\Box A$. The $\mathrm{Id}_\Box$ rule is the identity rule for boxed variables, which states that if we know that x is a variable of type $\Box A$, we can conclude that $\varepsilon(x)$ is of type A. The only way to add variables to the context is by using the $\to$-I rule. These variables are always boxed, since they are the parameters of abstractions. Due to the grammar of λ_b, all variables in λ_b occur inside the unboxing operator ε and thus the $\mathrm{Id}_\Box$ rule covers all cases. The $\to$-E rule states that if we have a term M of type $B \to A$ and a term N of type B, we can conclude that the application MN has type A. It is important that the argument N is a boxed term, which is why we denote its type as B.

Definition 3.5. Typing rules of λ_b

$$\frac{}{\Gamma, x : \Box A \vdash \varepsilon(x) : A}\ \mathrm{Id}_\Box$$

$$\frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x.M : A \to B}\ \to\text{-I}$$

$$\frac{\Gamma \vdash M : B \to A \qquad \Gamma \vdash N : B}{\Gamma \vdash MN : B}\ \to\text{-E}$$

$$\frac{\Gamma \vdash M : A}{\Gamma \vdash \mathsf{box}(M) : \Box A}\ \Box\text{-I}$$

We now demonstrate these typing rules with Example 3.6, where we show that the term $(\lambda x.\varepsilon(x))\mathsf{box}(\lambda y.\varepsilon(y))$ is well-typed. Note that the argument $\mathsf{box}(\lambda y.\varepsilon(y))$ is boxed. If the argument was not boxed, we would not be able to apply the abstraction $\lambda x.\varepsilon(x)$ to it.

Example 3.6. Well-typedness of $(\lambda x.\varepsilon(x))\mathsf{box}(\lambda y.\varepsilon(y))$

$$\dfrac{\dfrac{\dfrac{}{x : \Box(\Box A \to A) \vdash \varepsilon(x) : \Box A \to A}\ \mathrm{Id}_\Box}{\vdash \lambda x.\varepsilon(x) : \Box(\Box A \to A) \to (\Box A \to A)}\ \to\text{-I} \qquad \dfrac{\dfrac{\dfrac{}{y : \Box A \vdash \varepsilon(y) : A}\ \mathrm{Id}_\Box}{\vdash \lambda y.\varepsilon(y) : \Box A \to A}\ \to\text{-I}}{\vdash \mathsf{box}(\lambda y.\varepsilon(y)) : \Box(\Box A \to A)}\ \Box\text{-I}}{\vdash (\lambda x.\varepsilon(x))\mathsf{box}(\lambda y.\varepsilon(y)) : \Box A \to A}\ \to\text{-E}$$

3.2 Substitution and Free and Bound variables in λ_b

Since variables always occur inside ε, we need to redefine free and bound variables and substitution. For λ_b, we have:

Definition 3.7 (Free and Bound Variables)**.** The free and bound variables of a term M are defined by:

$$\begin{array}{llll} FV_b(\varepsilon(x)) & = \{\varepsilon(x)\} & BV(\varepsilon(x)) & = \emptyset \\ FV_b(\lambda x.M) & = FV(M)\backslash\{\varepsilon(x)\} & BV(\lambda x.M) & = BV(M) \cup \{\varepsilon(x)\} \\ FV_b(MN) & = FV(M) \cup FV(N) & BV(MN) & = BV(M) \cup BV(N) \end{array}$$

Definition 3.8 (Substitution in λ_b)**.** The substitution of a term Q for a variable $\varepsilon(x)$ in a term M is defined as follows:

$$\begin{aligned}
\varepsilon(x)[\varepsilon(x) \mapsto Q] &= Q \\
\varepsilon(y)[\varepsilon(x) \mapsto Q] &= \varepsilon(y) \text{ if } (x \neq y) \\
\mathsf{box}(M)[\varepsilon(x) \mapsto Q] &= \mathsf{box}(M[\varepsilon(x) \mapsto Q]) \\
(MN)[\varepsilon(x) \mapsto Q] &= M[x \mapsto Q]N[x \mapsto Q] \\
(\lambda x.M)[\varepsilon(x) \mapsto Q] &= \lambda x.M \\
(\lambda y.M)[\varepsilon(x) \mapsto Q] &= \lambda z.M[\varepsilon(y) \mapsto \varepsilon(z)][\varepsilon(x) \mapsto Q] \text{ if } (x \neq y)
\end{aligned}$$

with z a variable defined by:

1. If $\varepsilon(x) \notin FV_b(N)$ or $\varepsilon(y) \notin FV_b(M)$ then $z = y$
2. Otherwise, z can be any variable such that $\varepsilon(z) \notin FV(N)$ or $\varepsilon(z) \notin FV(M)$

With this new lambda calculus, it may not be completely clear that substitution preserves well-typedness. We can prove that substitution preserves well-typedness in λ_b.

Proposition 3.9 (Preservation of well-typedness)**.**

$$\Gamma, x : \Box A \vdash M : B \text{ and } \Gamma \vdash Q : A \implies \Gamma \vdash M[\varepsilon(x) \mapsto Q] : B$$

Proof. We prove by structural induction on the term M that this substitution preserves well-typedness.

1. $$\varepsilon(x)[\varepsilon(x) \mapsto Q] = Q$$

   Since Q is well-typed, and Q is the result of the substitution, we have that well-typedness is preserved.

2. $$\varepsilon(y)[\varepsilon(x) \mapsto Q] = \varepsilon(y) \quad \text{if } x \neq y$$

   We have that $x \neq y$, and the result of the substitution is $\varepsilon(y)$, which is well-typed.

3. $$\mathsf{box}(M)[\varepsilon(x) \mapsto Q] = \mathsf{box}(M[\varepsilon(x) \mapsto Q])$$

   We have that $\mathsf{box}(M)$ is well-typed and thus M is well-typed because of the $\Box$-I rule. By the inductive hypothesis, we have that $M[\varepsilon(x) \mapsto Q]$ is well-typed. We also have that $\mathsf{box}(M[\varepsilon(x) \mapsto Q])$ is well-typed because of the $\Box$-I rule. Thus, we have that $\mathsf{box}(M)[\varepsilon(x) \mapsto Q]$ is well-typed.

4.
$$(MN)[\varepsilon(x) \mapsto Q] = (M[\varepsilon(x) \mapsto Q])\ (N[\varepsilon(x) \mapsto Q])$$

We have that MN is well-typed and thus M and N are well-typed. By the inductive hypothesis, we get that $M[\varepsilon(x) \mapsto Q]$ is well-typed, and $N[\varepsilon(x) \mapsto Q]$ is well-typed. We also have that application of two well-typed terms is well-typed, so we have that $(MN)[\varepsilon(x) \mapsto Q]$ is well-typed.

5.
$$(\lambda x.M)[\varepsilon(x) \mapsto Q] = \lambda x.M$$

We have that $\lambda x.M$ is well-typed and thus M is well-typed. Since $\varepsilon(x)$ is a bound variable in $\lambda x.M$, no substitution occurs and the result is the well-typed term $\lambda x.M$.

6.
$$(\lambda y.M)[\varepsilon(x) \mapsto Q] = \lambda z.(M[\varepsilon(y) \mapsto \varepsilon(z)][\varepsilon(x) \mapsto Q]) \quad \text{if } x \neq y$$

We have that $\lambda y.M$ is well-typed and thus M is well-typed. By the inductive hypothesis, we have that $M[\varepsilon(y) \mapsto \varepsilon(z)]$ and $M[\varepsilon(y) \mapsto \varepsilon(z)][\varepsilon(x) \mapsto Q]$ are well-typed. We also have that abstraction of a well-typed term is well-typed, so we have that
$\lambda z.(M[\varepsilon(y) \mapsto \varepsilon(z)][\varepsilon(x) \mapsto Q])$ is well-typed. □

With this definition of substitution, we define the reduction rule for λ_b in Definition 3.10. In λ_b, the arguments of abstractions are always boxed, so we can only apply abstractions to boxed terms, which makes sense since the parameters like x are always boxed. Since variables only occur insides ε, we can replace those, including ε, with the corresponding unboxed term.

Definition 3.10 (β_b)**.** In λ_b, β reduction is defined as follows:

$$(\lambda x.M)\mathsf{box}(N) \to M[\varepsilon(x) \mapsto N] \qquad (\beta_b)$$

For completeness, we need to define a new closure rule. This new rule only applies to the λ_b, and allows reduction under boxed terms. We call this rule ζ, and Definition 3.11 defines it.

Definition 3.11. ζ

$$\frac{M \to_{\beta_b} N}{\mathsf{box}(M) \to_{\beta_b} \mathsf{box}(N)}\ (\zeta)$$

The relation $\rightarrow_{\beta_b}$ is the relation that allows reduction under all term formers, including boxes. We also define three other relations: β_b closed under μ and ν, β_b closed under μ and β_b closed under ν. We call them $\rightarrow_{we}$, $\rightarrow_{b_n}$ and $\rightarrow_{b_v}$ respectively. Table 3.1 is an overview of all relations. The relation $\rightarrow_{we}$ is weak since values do not reduce. That is, abstractions and terms of the form $\varepsilon(x)$ do not reduce. The relation $\rightarrow_{we}$ is also external as boxes do not reduce. This relation is called call-by-box (cbb) evaluation. We will see after defining the embeddings that $\rightarrow_{we}$ collapses to $\rightarrow_{b_n}$ in Girard's embedding and that $\rightarrow_{we}$ collapses to $\rightarrow_{b_v}$ in Gödel's embedding.

Relation	$\boldsymbol{\beta}$	$\boldsymbol{\mu}$	$\boldsymbol{\nu}$	$\boldsymbol{\nu_<}$	$\boldsymbol{\xi}$	$\boldsymbol{\zeta}$
$\rightarrow_{\beta_n}$	β_n	✓	✓	✓	✓	
$\rightarrow_n$	β_n	✓				
$\rightarrow_w$	β_n	✓	✓			
$\rightarrow_{\beta_v}$	β_v	✓	✓	✓	✓	
$\rightarrow_v$	β_v	✓		✓		
$\rightarrow_{\beta_b}$	β_b	✓	✓	✓	✓	✓
$\rightarrow_{we}$	β_b	✓	✓	✓		
$\rightarrow_{b_n}$	β_b	✓				
$\rightarrow_{b_v}$	β_b		✓			

Table 3.1: Closure Rules for Different Reduction Relations

Chapter 4

Embeddings in λ_b

4.1 Girard's Embedding

In this section, we present Girard's embedding, which is a method to translate intuitionistic logic into linear logic [5]. Jean-Yves Girard introduced this embedding to represent intuitionistic logic in a linear framework, where resources are managed explicitly. Linear logic can be seen as a refinement of classical logic, where assumptions - or resources - are managed explicitly. In particular, linear logic distinguishes between assumptions that should be used exactly once and those that can be used any number of times, including zero times. However, this thesis does not focus on linear logic, but on the way Girard's embedding can be used to represent the call-by-name λ-calculus in the modal call-by-box λ-calculus, λ_b.

In general, Girard's embedding is not limited to linear logic. It is more general technique to represent one logical system in a different framework. The embedding can be used to represent one logical system in another framework that has a way to manage resources or assumptions. In the context of λ_n, parameters of functions can be seen as resources that are consumed when the function is applied. These resources can be used as many times as needed, hence the connection with Girard's embedding and linear logic.

4.2 Embedding of λ_n into λ_b

The model embedding of the λ_n into the λ_b is given by Girard's translation, denoted as $(\cdot)^\circ$. Thus, M° is the translation of a term M from the λ_n to the λ_b, and A° is the translation of a type A from the λ_n to the λ_b. The translation is inductively defined in Figure 4.1 for both terms and types. We also give the translation of λ-term $\mathbf{I}x$ in Example 4.1.

$$
\begin{array}{rr}
X^\circ = X & x^\circ = \varepsilon(x) \\
(A_1 \to A_2)^\circ = \Box A_1{}^\circ \to A_2{}^\circ & (\lambda x.M)^\circ = \lambda x.M^\circ \\
 & (MN)^\circ = M^\circ \mathsf{box}(N^\circ)
\end{array}
$$

Figure 4.1: Girard's translation from λ_n to λ_b.

Example 4.1. $(\mathbf{I}x)^\circ$

$$
\begin{aligned}
(\mathbf{I}x)^\circ &= (\lambda x.x)^\circ \mathsf{box}(x^\circ) \\
&= (\lambda x.x^\circ)\mathsf{box}(x^\circ) \\
&= (\lambda x.\varepsilon(x))\mathsf{box}(\varepsilon(x)).
\end{aligned}
$$

From Girard's translation, we see that a boxed λ-term can never occur on its own. Boxed terms are always part of an application, where the left-hand side is a term in λ_b and the right-hand side is the boxed term. Therefore, the image of Girard's translation is a subset of the λ_b given by the following grammar:

$$
M, N \quad ::= \quad \varepsilon(x) \mid \ \lambda x.M \mid \ M\mathsf{box}(N)
$$

We can reason that the relation $\to_{we}$ collapses to $\to_{b_n}$ when we consider the image of Girard's translation. Since application in Girard's image is always of the form $M\mathsf{box}(N)$, the right-hand side of the application can never be reduced without the use of the ζ rule. In $\to_{we}$, the only closure rules allowed are μ and ν. It is clear that the ν rule can never be applied in Girard's image as it is not possible to reduce under boxed terms. Thus, the only closure rule that can be applied is the μ rule, and $\to_{we}$ collapses to $\to_{b_n}$ in Girard's image.

We give a small example to clarify everything. Consider the λ-term $\mathbf{I}x$. It holds that $\mathbf{I}x \to_n x$ in λ_n. Example 4.2 shows that $(\mathbf{I}x)^\circ \to_{b_n} x^\circ$ also holds.

Example 4.2. $(\mathbf{I}x)^\circ$ under $\to_{b_n}$

$$
\begin{array}{rcl}
(\mathbf{I}x)^\circ & = & (\lambda x.x)^\circ \mathsf{box}(x^\circ) \\
 & = & (\lambda x.x^\circ)\mathsf{box}(x^\circ) \\
 & = & (\lambda x.\varepsilon(x))\mathsf{box}(\varepsilon(x)) \\
 & \to_{b_n} & (\lambda x.\varepsilon(x))[\varepsilon(x) \mapsto \varepsilon(x)] \\
 & = & \varepsilon(x) \\
 & = & x^\circ
\end{array}
$$

4.3 Gödel's Embedding

In this section, we present Gödel's embedding, introduced by Kurt Gödel. Gödel introduced this embedding as a method to translate intuitionistic logic into modal logic S4 [7]. More specifically, an embedding to represent intuitionistic logic in a modal framework, where box modality, $\Box$ is used to indicate provability. That is, $\Box P$ can be read as P is provable.

In general, Gödel's embedding is used to represent one more restricted logical system in a stronger framework. This thesis uses Gödel's embedding to represent the call-by-name λ-calculus in the modal call-by-box λ-calculus, λ_b. The intuition behind Gödel's embedding is that boxes are used to represent values, which are already evaluated expressions.

4.4 Embedding of λ_n into λ_b

In the model embedding of λ_v into λ_b, we make use of an abbreviation, which we call raise. The idea of raise is to swap two lambda terms, while removing a box of one λ-term. Its use will become clear later in this section when we give some examples. We define raise as follows:

$$\mathsf{raise}(M) := \lambda z.\varepsilon(z)M$$

The model embedding of λ_v into λ_b is given by Gödel's translation, denoted as $(\cdot)^*$. Thus, M^* is the translation of a term M from λ_v to λ_b, and A^* is the translation of a type A from the λ_v to the λ_b. We also define $(\cdot)^\bullet$ for the translation of values and types in λ_v. The embedding $(\cdot)^*$ and $(\cdot)^\bullet$ are mutually inductively defined in Figure 4.2 for both terms and types. That is, we define $(\cdot)^*$ in terms of $(\cdot)^\bullet$ and vice versa. We also give the translation of λ-term $\mathbf{I}x$ in Example 4.3.

$$\begin{aligned} A^* &= \Box A^\bullet & V^* &= \mathsf{box}(V^\bullet) \\ & & (MN)^* &= \mathsf{raise}(N^*)M^* \end{aligned}$$

$$\begin{aligned} X^\bullet &= X & x^\bullet &= \varepsilon(x) \\ (A_1 \to A_2)^\bullet &= \Box A_1{}^\bullet \to \Box A_2{}^\bullet & (\lambda x.M)^\bullet &= \lambda x.M^* \end{aligned}$$

Figure 4.2: Gödel's translation from λ_v to λ_b.

Example 4.3. $(\mathbf{I}x)^*$

$$\begin{aligned}(\mathbf{I}x)^* &= \mathsf{raise}(x^*)(\lambda x.x)^* \\ &= \mathsf{raise}(\mathsf{box}(x^\bullet))\mathsf{box}((\lambda x.x)^\bullet) \\ &= \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.x^*) \\ &= \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.\mathsf{box}(x^\bullet)) \\ &= \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.\mathsf{box}(\varepsilon(x)))\end{aligned}$$

Let us now reason about the image of Gödel's translation. From the translation, it follows that all applications in the image of Gödel's translation are of the form $\mathsf{raise}(M')M$, where both M' and M are terms in λ_b. Every term in the image that is not an application, is of the form $\mathsf{box}(V)$, with V a value in λ_b. Thus, the image of Gödel's translation is a subset of λ_b given by the following grammar:

$$M \quad ::= \quad \mathsf{box}(V) \mid \mathsf{raise}(M')M \qquad\qquad V \quad ::= \quad \varepsilon(x) \mid \lambda x.M$$

Now, we reason that the relation $\to_{we}$ collapses to $\to_{b_v}$ when we consider the image of Gödel's translation. We do this by showing that the only closure rule that can be applied on each type of term in Gödel's image is the ν rule.

1. If a term in Gödel's image is of the form $\mathsf{box}(V)$, we cannot apply any reduction rule, since we do not allow reductions under boxed terms. Besides, V is a value, which cannot be reduced.

2. If a term in Gödel's image is of the form $\mathsf{raise}(M')M$, we can only apply the ν rule. We have that $\mathsf{raise}(M') = \lambda z.\varepsilon(z)M'$, which is a value. Thus, $\mathsf{raise}(M')$ cannot be reduced and the μ rule cannot be applied. The only closure rule that can be applied is the ν rule.

We give a small example to clarify everything. Consider the λ-term $\mathbf{I}x$. It holds that $\mathbf{I}x \to_v x$ in λ_n. Example 4.2 shows that $(\mathbf{I}x)^* \to_{b_v}^* x^*$ also holds.

Example 4.4. $(\mathbf{I}x)^*$

$$
\begin{aligned}
(\mathbf{I}x)^* &= \mathsf{raise}(x^*)(\lambda x.x)^* \\
&= \mathsf{raise}(\mathsf{box}(x^\bullet))\mathsf{box}((\lambda x.x)^\bullet) \\
&= \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.x^*) \\
&= \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.\mathsf{box}(x^\bullet)) \\
&= \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.\mathsf{box}(\varepsilon(x))) \\
&= (\lambda z.\varepsilon(z)\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.\mathsf{box}(\varepsilon(x))) \\
&\to_{b_v} (\lambda x.\mathsf{box}(\varepsilon(x)))\mathsf{box}(\varepsilon(x)) \\
&\to_{b_v} \mathsf{box}(\varepsilon(x)) \\
&= \mathsf{box}(x^\bullet) \\
&= x^*
\end{aligned}
$$

Chapter 5

Formalisation in Agda

5.1 Intro to Agda

This section, aims to provide a brief introduction to Agda, a dependently typed functional programming language and proof assistant. Agda uses strong typing and dependent types, and thus it can be used to write proofs constructively, which can be run as programs. In order to understand the formalisation of λ_b in Agda, we discuss the following basic concepts of Agda: data types, parameterised data types, functions, dependent data types, implicit arguments, and hole-based programming.

Data types, can be seen as a collection of values that have something in common. This collection is usually structured in some kind of way, such as a list or a tree. Elements of a data type can be constructed using constructors, which are functions that return a value of the data type. In Agda, we can create a data type with the data keyword, followed by the name of the data type and its constructors.

Example 5.1 shows an example of a simple inductive data type: the natural numbers. It defines two constructors:

- zero, representing the base case (the number zero),
- suc, which takes a natural number and returns its successor.

The keyword Set in the type signature indicates the type of type. This is analogous to * in Haskell, which denotes the kind of types.

Example 5.1. Data type in Agda:

```
data ℕ : Set where
  zero : ℕ
  suc : ℕ → ℕ
```

Data types in Agda can also have parameters. This makes them more flexible and allows for the creation of more complex data types. Example 5.2 shows an example of a parameterised data type, which is a list that can hold elements of any type A. We could make a list of natural numbers, which will have the type List $\mathbb{N}$, or a list of Booleans, which will have the type List Bool. Note that the parameter A occurs before the colon, indicating that it is a type parameter.

Example 5.2. Parameterised data type in Agda:

```
data List (A : Set) : Set where
  [] : List A
  _::_ : A → List A → List A
```

Functions in Agda are defined using pattern matching, similar to Haskell. Each function has a type signature that defines the type of the function and its arguments. After the type signature, a number of equations follows, defining the function for each pattern, where functions can be recursive. Example 5.3 shows such a function, which doubles a natural number.

Functions can also be higher order, taking other functions as arguments or returning functions as results. Function application of function F to argument x is written as $F\ x$ in Agda. Infix functions are denoted with underscores, such as _ :: _ in Example 5.2, which is the constructor for the list data type. The first argument can be put on the left side of the constructor and the second argument on the right side, such as in $x :: xs$.

Example 5.3. Simple Recursive Function in Agda:

```
double : ℕ → ℕ
double zero = zero
double (suc n) = suc (suc (double n))
```

Example 5.4 shows an example of a dependent data type, which is a vector that has a length that is encoded in its type. In this example, we see that the type of the vector depends on a natural number, which is different for both constructors. The dependent type occurs after the colon, which is $\mathbb{N}$ in this case. Dependent types differ from parameterised types in that dependent types can differ for each constructor, while parameterised types have a single type parameter that is shared across all constructors.

The constructor _ :: _ in Example 5.4 is also an example of a dependent function. That is, the return type of _ :: _ depends on the value of the implicit argument n. The argument n is implicit, denoted by the curly braces around it.

We see that [] constructs a vector of length zero, resulting in type Vector A zero, while the constructor _ :: _ constructs a vector of length $n+1$,

resulting in type Vector A (suc n). Note that the implicit argument n can be inferred from the input type Vector A n.

Example 5.4. Dependent data type in Agda:

```
data Vector (A : Set) : ℕ → Set where
  [] : Vector A zero
  _::_ : {n : ℕ} → A → Vector A n → Vector A (suc n)
```

5.1.1 Hole-Based Programming

Agda is made to be used with the Emacs text editor, and the combination of Agda and Emacs provides a way to construct definitions and proofs interactively, which is called hole-based programming. Terms can be constructed step by step, using question marks as placeholders, which will turn into holes, indicating incomplete definitions. It is best to show this with an example. Note that we could have used more holes in Example 5.5, but we feel like that this example is sufficient to understand the concept of hole-based programming.

Example 5.5. Hole-based programming in Agda:
This example defines the function plus that adds two natural numbers. It defines the function with a question mark, which we can fill in later. The type signature is already given.

```
plus : ℕ → ℕ → ℕ
plus = ?
```

Pressing the keys C-c C-l (while pressing the control key, typing c and then typing l) will load the file and will transform the question mark into a hole.

```
plus : ℕ → ℕ → ℕ
plus = {!   !}
```

We can now let Agda give the arguments of the plus function by pressing C-c C-c. You will get the following output:

```
pattern variables to case (empty for split on result):
```

Since we do not have any arguments yet, we can press enter.

```
plus : ℕ → ℕ → ℕ
plus x x₁ = {!   !}
```

Now, we would like to do a case distinction on the first argument. We let Agda do this by pressing C-c C-c again. The same prompt will be shown as before, but now we type the first argument, which is x in this case.

```
plus : ℕ → ℕ → ℕ
plus zero x₁ = {!   !}
plus (suc x) x₁ = {!   !}
```

Inside the first hole, we press C-c C-, . This gives us information about the type of the hole and all the variables in scope, which are the variables that we can use to fill in the hole. We get something like the following output:

```
Goal: ℕ
------------------------
x₁ : ℕ
```

This tells us we need to produce a natural number and that we can use the variable x_1 inside this hole. Since we have that $0 + x_1 = x_1$, we fill in the hole with x_1, which can be done by typing x_1 inside the hole and pressing C-c C-space. The hole will be filled in and Agda will check if the type of the hole is correct. Pressing C-c C-f will move the cursor forward to the next hole.

```
plus : ℕ → ℕ → ℕ
plus zero x₁ = x₁
plus (suc x) x₁ = {!   !}
```

Typing C-c C-, in this second hole will give us the type of the next hole. The output will be similar to the following:

```
Goal: ℕ
------------------------
x₁ : ℕ
x : ℕ
```

The goal is to produce a natural number. It is the result of an addition. Since one of the arguments is suc x, we know the result is not zero. The result should a natural number that uses the suc constructor. We fill in the hole with suc ? and type C-c C-space. We closed a hole and made a new hole, which is the argument of the suc constructor. We should now have the following code:

```
plus : ℕ → ℕ → ℕ
plus zero x₁ = x₁
plus (suc x) x₁ = suc {!   !}
```

We can fill in the last hole with plus x x_1 and press C-c C-space to close the hole. The final code will look like this:

```
plus : ℕ → ℕ → ℕ
plus zero x₁ = x₁
plus (suc x) x₁ = suc (plus x x₁)
```

Example 5.5 shows how to use the commands C-c C-l, C-c C-c, C-c C-f, C-c C-, , and C-c C-space to interactively develop a function in Agda. However, there many more commands available in Agda to help with hole-based programming (refer to the Agda documentation for more information).

5.2 Challenges in Formalisation

5.2.1 De Bruijn Indices

De Bruijn indeces, introduced by Nico G. de Bruijn, are a method to eliminate the need for variable names in the λ-calculus [4]. This is useful because it makes the notion of α-equivalence irrelevant. We do not have to worry about variables with the same name that refer to different variables, or about variable capture, simplifying the formalisation in Agda. The definition of substitution becomes simpler because of this. This subsection first explains the concept of de Bruijn indices, and then shows how they are implemented in the formalisation.

The λ-terms that use de Bruijn indices do not use variable names, but integers. We precede the integers with # to prevent confusion if the index is higher than 9. Without #, 10 could be confused with two indices 1 and 0 that occur next to each other. These integers represent the "distance" to the corresponding λ. Example 5.6 gives representations of λ-terms without free variables, using de Bruijn indices. These terms are said to be closed.

Example 5.6. Closed λ-terms represented with **de Bruijn indices**:

$\lambda x.x \equiv \lambda.\#0$
$\lambda x.\lambda y.x \equiv \lambda.\lambda.\#1$
$\lambda x.\lambda y.y \equiv \lambda.\lambda.\#0$
$\lambda x.x\lambda y.y \equiv \lambda.\#0\lambda.\#0$

Representing open λ-terms (λ-terms with free variables) using de Bruijn indices is a little more complex. First, we represent the λ-term with de Bruijn indices, ignoring the free variables. Then, we give each free variable a unique index, that is larger than the largest index of the bound variables. **??** will demonstrate this.

Example 5.7. Open λ-terms represented with **de Bruijn indices**:

$x\lambda y.y \equiv x\lambda.0 \equiv \#1\lambda.\#0$
$x\lambda y.x \equiv x\lambda.x \equiv \#1\lambda.\#1$
$\lambda x.\lambda t.xt(\lambda s.w)wy \equiv \lambda.\lambda.\#1\#0(\lambda.w)wy \equiv \lambda.\lambda.\#1\#0(\lambda.\#4)\#3\#2$

In the formalisation, we will represent de Bruijn indices similar to how natural numbers are represented in Agda. De Bruijn index 0 is represented as `Z`, de Bruijn index 1 as `S Z`, de Bruijn index 2 as `S (S Z)`, and so on.

5.2.2 Well-typed Terms

This subsection discusses the restriction to well-typed terms in the formalisation of λ_b. The restriction to well-typed terms is important because it avoids all complications that arise from ill typed terms. For example, in the context of evaluation, it makes sense to restrict to well-typed terms, because ill-typed terms do not have a normal form. Evaluation is only defined for well-typed terms, and thus the restriction to well-typed terms aligns with the goal of the thesis.

In the formalisation, we use an intrinsically typed approach where terms are defined together with their types. So λ-terms in the formalisation are automatically associated with a type, and we can only construct well-typed terms. This approach is also referred to as Church style typing, and was introduced by Alonzo Church in [3].

The formalisation is based on the representation proposed by Thorsten AltenKirch and Bernhard Reus in [1]. We briefly go over the types and λ-terms in the formalisation, and how they are defined. Definition 5.8 gives the formal definition of types in the λ-calculus.

Definition 5.8. Types in the λ-calculus:

```
data Type : Set where
  _⇒_ : Type → Type → Type
  X : Type
```

Terms in the λ-calculus are defined together with a context, which can be seen as a list of Types. We do not need to name the variables in the context, because we use de Bruijn indices to refer to them. Definition 5.9 gives the formal definition of contexts in the λ-calculus. This construction has roughly the same idea as the list data type that we gave as example in Example 5.2.

Definition 5.9. Context in the λ-calculus:

```
data Context : Set where
  ∅ : Context
  _,_ : Context → Type → Context
```

We now define de Bruijn indices that refer to the variables in the context. They can be seen as a proof that some type is in the context. If A occurs in the context, then A ∈ Γ denotes that A is in the context Γ, and the Bruijn index is the proof of this. The de Bruijn index Z refers the most recently introduced variable in the context, which appears on the right most position of the context. The de Bruijn index S Z refers to the second most recently introduced variable, and so on.

Definition 5.10. De Bruijn indices in the λ-calculus:

```
data _∈_ : Type → Context → Set where
  Z : ∀ {Γ A} → A ∈ (Γ , A)
  S : ∀ {Γ A B} → A ∈ Γ → A ∈ (Γ , B)
```

Definition 5.11 gives the formal definition of terms in the λ-calculus. The λ-terms are defined by the context they are in and the type they have. For example, Γ ⊢ A denotes the λ-terms that have type A in the context Γ. With the constructors ‘, ƛ and ·, we can construct λ-terms.’ The ‘ constructor is used to construct λ-terms that are variables if we can provide a proof that the variable is in the context. Abstractions are constructed with the ƛ constructor, which takes a context with more than one variable, and creates a new λ-term that has one less variable in the context with one. This new λ-term is the abstraction of the original λ-term. The · constructor is used to construct applications of λ-terms. Given an abstraction and an argument for that abstraction, · constructs a new λ-term of the return type of the abstraction. Examples of λ-terms are given in Example 5.12. Note that we need to give the type of the λ-term, which is of the form Γ ⊢ A, and that we provide evidence for this type when constructing the λ-term.

Definition 5.11. Terms in the λ-calculus:

```
data _⊢_ : Context → Type → Set where
  ‘ : ∀ {Γ A}
    → A ∈ Γ
      -------
    → Γ ⊢ A

  ƛ : ∀ {Γ A B}
    → Γ , A ⊢ B
      --------------
```

```
      → Γ ⊢ A ⇒ B

    _·_ : ∀ {Γ A B}
      → Γ ⊢ A ⇒ B
      → Γ ⊢ A
        --------------
      → Γ ⊢ B
```

Example 5.12. λ-terms in the λ-calculus:

```
t₁ : ∅ , X ⊢ X
t₁ = ‘ Z
-- same as x

t₂ : ∅ ⊢ X ⇒ X
t₂ = ƛ (‘ Z)
-- same as λx.x

t₃ : ∅ , X ⊢ X
t₃ = ƛ (‘ Z) · ‘ Z
-- same as (λx.x)y

t₄ : ∅ , X ⇒ X ⊢ X ⇒ X
t₄ = ƛ (‘ (S Z) · (‘ (S Z) · ‘ Z))
-- same as λx.y(yx)

t₅ : ∅ , X ⇒ X , X ⊢ X ⇒ X
t₅ = ƛ (‘ (S (S Z)) · (‘ (S (S Z)) · ‘ Z))
-- same as λx.y(yx), but with a different context
```

Now that we have defined the types, contexts and terms in the standard λ-calculus, we define the λ-terms in the λ_b. Definition 5.13 gives the formal definition of λ-terms in the λ_b. Note that b is the suffix of many constructors, which is to distinguish between the constructors of the standard λ-calculus and λ_b.

Definition 5.13. Terms of λ_b in Agda:

```
data Typeb : Set
data BoxedTypeb : Set

data Typeb where
  _⇒b_ : BoxedTypeb → Typeb → Typeb
  X : Typeb
  Box : BoxedTypeb → Typeb
```

```
data BoxedTypeb where
  □ : Typeb → BoxedTypeb

BOX : Typeb → Typeb
BOX A = Box (□ A)

data Contextb : Set where
  ∅b : Contextb
  _,b_ : Contextb → BoxedTypeb → Contextb

data _∈b_ : BoxedTypeb → Contextb → Set where
  Zb : ∀ {Γ □A} → □A ∈b (Γ ,b □A)
  Sb : ∀ {Γ □A □B} → □A ∈b Γ → □A ∈b (Γ ,b □B)

data _⊢b_ : Contextb → Typeb → Set where
  ε : ∀ {Γ A}
    → □ A ∈b Γ
      -------
    → Γ ⊢b A

  box : ∀ {Γ A}
    → Γ ⊢b A
    --------------
    → Γ ⊢b BOX A

  ƛb : ∀ {Γ A B}
    → Γ ,b □ A ⊢b B
      ---------------
    → Γ ⊢b □ A ⇒b B

  _·b_ : ∀ {Γ A B}
    → Γ ⊢b □ A ⇒b B
    → Γ ⊢b BOX A
      --------------
    → Γ ⊢b B
```

5.2.3 Substitution

The next challenge in the formalisation is a correct definition of substitution. This definition appeared to be the most complex part of the formalisation, and it caused the most problems with the proofs later in this thesis. The definition of substitution is similar to the definition of substitution that is used in the chapter "De Bruijn indices" in the PLFA book [8]. This definition is based on the definition by Conor McBride [2], which gives a

similar definition of substitution in Rocq. In Definition 5.14, we only give the definition of substitution in Agda of λ-terms in λ_b, since the definition of substitution in λ-calculus is exactly the same as [8]. We first define parallel substitution, which is the substitution of a list of λ-terms by another list of λ-terms. Then we define single substitution, which might be more familiar and is the substitution of a single λ-term by another λ-term. Single substitution of the λ-term that occurs most recently in the context of M by λ-term N is denoted by $M[N]b$ in the formalisation of λ_b.

Definition 5.14. Substitution in Agda of λ_b:

```
extb : ∀ {Γ Δ}
  → (∀ {A} → A ∈b Γ → A ∈b Δ)
    ----------------------
  → (∀ {A B} → A ∈b Γ ,b B → A ∈b Δ ,b B)
extb f Zb = Zb
extb f (Sb x) = Sb (f x)

renameb : ∀ {Γ Δ}
  → (∀ {A} → A ∈b Γ → A ∈b Δ)
    ----------------------
  → (∀ {A} → Γ ⊢b A → Δ ⊢b A)
renameb f (ε x) = ε (f x)
renameb f (box N) = box (renameb f N)
renameb f (ƛb M) = ƛb (renameb (extb f) M)
renameb f (M ·b N) = renameb f M ·b (renameb f N)

exts : ∀ {Γ Δ}
  → (∀ {A} → □ A ∈b Γ → Δ ⊢b A)
    ----------------------
  → (∀ {A B} → □ A ∈b Γ ,b B → Δ ,b B ⊢b A)
exts f Zb = ε Zb
exts f (Sb x) = renameb Sb ( f x )

substb : ∀ {Γ Δ}
  → (∀ {A} → □ A ∈b Γ → Δ ⊢b A)
    ------------------------
  → (∀ {A} → Γ ⊢b A → Δ ⊢b A)
substb f (ε x) = f x
substb f (box N) = box (substb f N)
substb f (ƛb M) = ƛb (substb (exts f) M)
substb f (M ·b N) = substb f M ·b (substb f N)

fb : ∀ {Γ C} (M : Γ ⊢b C) → ∀ {A} → □ A ∈b Γ ,b □ C → Γ ⊢b A
fb M Zb = M
```

```
fb M (Sb x) = ε x

_[_]b : ∀ {Γ A C}
  → Γ ,b □ C ⊢b A
  → Γ ⊢b C
    -------------
  → Γ ⊢b A
_[_]b N M = substb (fb M) N
```

We give some examples of this definition of substitution in λ_b in Example 5.15. In the example, we have that $\mathsf{t}_1\ [\ \mathsf{t}_2\ ]\mathsf{b} \equiv\ \mathsf{t}_3$

Example 5.15. Substitution in λ_b:

```
t1 : ∅b ,b □ (□ X ⇒b X) ⊢b □ X ⇒b X
t1 = ƛb (ε (Sb Zb) ·b box (ε (Sb Zb) ·b box (ε Zb)))

t2 : ∅b ⊢b □ X ⇒b X
t2 = ƛb (ε Zb)

t3 : ∅b ⊢b □ X ⇒b X
t3 = ƛb (ƛb (ε Zb) ·b box (ƛb (ε Zb) ·b box (ε Zb)))

_ : t1 [ t2 ]b ≡ t3
_ = refl
```

5.2.4 Weakening and Renaming

Weakening and renaming becomes something to take into consideration in the formalisation. Two λ-terms that are the same except for the contexts are not exactly the same. An example to demonstrate this is the formal definition of raise in Agda, where the use of renameb is required. Recall that raise was defined as follows:

$$\mathsf{raise}(M) := \lambda z.\varepsilon(z)M$$

The term M outside the function (in $\mathsf{raise}(M)$) has a different context than the term M inside the function of raise ($\lambda z.\varepsilon(z)M$). We attempt to give a typing derivation in Figure 5.1. We assume that term M has type B' which can be derived from Γ. However, we are stuck in the derivation since we do not have rules to discard assumptions. We see that we need to proof $\Gamma, z : \Box(\Box B \to B') \vdash M : B'$ instead of $\Gamma \vdash M : B'$.

$$\dfrac{\dfrac{}{\Gamma, z : \Box\Box B \to B' \vdash \varepsilon(z) : \Box B \to B'}\,\mathrm{Id}_{\Box} \qquad \begin{array}{c}\vdots \\ \Gamma, z : \Box\Box B \to B' \vdash M : B'\end{array}}{\Gamma, z : \Box\Box B \to B' \vdash \varepsilon(z)M : B'}\,{\to}\text{-E}$$
$$\dfrac{\Gamma, z : \Box\Box B \to B' \vdash \varepsilon(z)M : B'}{\Gamma \vdash \lambda z.\varepsilon(z)M : \Box(\Box B \to B') \to B'}\,{\to}\text{-I}$$

Figure 5.1: The attempt of the typing derivation of $\lambda z.\varepsilon(z)M$

However, it makes sense that $\Gamma \vdash M : B'$ implies $\Gamma, z : \Box(\Box B \to \Box B') \vdash M : \Box B'$. We only add an extra assumption that we do not use, so we need a function that can handle this. This is where renameb comes in. The function renameb "renames" the de Bruijn indices in the λ-terms with a certain function and changes the context accordingly. In our case, we need to rename the indices with Sb (S for λ_b).

Let us show how this works in the formalisation. We have a λ-term M that has type B' in context Γ, which is denoted by $M : \Gamma \vdash B'$ in the formalisation. Therefore, we use renameb in such a way that it returns a λ-term of the type $\Gamma\ ,\mathsf{b}\ \Box(\Box B \to B') \vdash \Box B'$. We achieve this by renaming with the function Sb as shown below. The resulting type is $\Gamma\ ,\mathsf{b}\ \Box(\Box B \to \Box B') \vdash \Box B'$

$$\mathsf{rename}\ (\mathsf{Sb}\ \{\Box B = \Box(\Box B \to \Box B')\})\ M$$

Here, we needed to give the implicit argument since Sb needs to know what to the resulting context should be. Now, we incorporate this into the definition of raise. Note that Agda can automatically infer the implicit arguments of Sb in the definition of raise.

Definition 5.16. raise in Agda

```
raise : ∀ {Γ A B}
  → (t : Γ ⊢b BOX B)
  → Γ ⊢b □ (□ B ⇒b BOX A) ⇒b BOX A
raise t = ƛb (ε Zb ·b renameb Sb t)
```

5.2.5 Helper Lemmas

This subsection goes over some helper lemmas needed to prove the main reduction properties of λ_b. The first lemma we discuss is the lemma pres-subst, which states that the order of embedding and substitution does not matter. So substitution is preserved and reflected under embedding. We prove this lemma by using another lemma, pres-simultaneous-subst, which states that simultaneous substitution is preserved and reflected under embedding. This

can be proven by induction on the structure of the λ-term in which substitution is done. However, in Definition 5.14, we defined substitution in terms of extsb, which is defined in terms of renameb and extb. Therefore, we also make the following helper lemmas: pres-ext, pres-rename and pres-exts.

We only show the lemmas pres-subst and pres-simultaneous-subst for Girard's embedding, since the idea for Gödel's translation is the same. Lemma 5.17 shows proof of the pres-subst lemma, and lemma 5.18 shows the proof of the pres-simultaneous-subst lemma. Lemma pres-subst-func is a helper lemma in lemma 5.17 that proofs assumption h in lemma 5.18. This assumption shows how functions fb' and f' are related to each other, which is needed to prove pres-simultaneous-subst.

Lemma 5.17. Preservation of single substitution:

```
-- Preservation and Reflection of
-- single substitution of Girard's translation
pres-subst : ∀ {Γ A B}
  → (M : Γ , B ⊢ A)
  → (N : Γ ⊢ B)
  → (embedTerm M) [ embedTerm N ]b ≡ embedTerm (M [ N ])
pres-subst M N =
  pres-simultaneous-subst (fb (embedTerm N)) (f N) (pres-subst-func N) M
```

Lemma 5.18. Preservation of simultaneous substitution:

```
-- Preservation and Reflection of
-- simultaneous substitution of Girard's translation
pres-simultaneous-subst : ∀ {Γ Δ A}
  → (fb' : ∀ {Ab} → □ Ab ∈b embedContext Γ → embedContext Δ ⊢b Ab)
  → (f' : ∀ {A} → A ∈ Γ → Δ ⊢ A)
  → (h : ∀ {A'} → (x : A' ∈ Γ) → fb' (embedLookup x) ≡ embedTerm (f' x) )
  → (M : Γ ⊢ A)
  → substb fb' (embedTerm M) ≡ embedTerm (subst f' M)
pres-simultaneous-subst fb' f' h (` x) =
  h x
pres-simultaneous-subst fb' f' h (ƛ M) =
  cong
    ƛb
    ( pres-simultaneous-subst (extsb fb') (exts f') (pres-exts fb' f' h) M )
pres-simultaneous-subst fb' f' h (M · M₁) =
  cong₂
    _·b_
    (pres-simultaneous-subst fb' f' h M)
    (cong box (pres-simultaneous-subst fb' f' h M₁))
```

Some other interesting helper lemmas are the congruence, compose and commute lemmas, which were needed in the poofs of Gödel's embedding.

This refined embedding makes use of the abbrevation raise, which is formally defined in Definition 5.16 with the use of renameb. This use of renameb makes the proofs a lot more complex and requires the need for these helper lemmas. We only give congruence, compose and commute helper lemma's for simultaneous substitution because these lemmas are similar for extsb, renameb and extb. Lemma 5.19 uses helper lemmas subst-renameb-comp and substb-rename-commute, of which subst-renameb-comp is given in lemma 5.20. The lemma substb-cong is used in lemma 5.19 and given in lemma 5.21.

Lemma 5.19. Commutativity of renaming followed by substitution under an extended context and substitution followed by renaming:

```
-- Renaming followed by substitution with an extended context (extsb)
-- commutes with substitution followed by renaming.
substb-exts-rename-Sb-commute : ∀ {Γ A B Δ}
  → (fb' : ∀ {Ab} → □ Ab ∈b Γ → Δ ⊢b Ab)
  → (t : Γ ⊢b A)
  → substb (extsb fb' {B = B}) (renameb Sb t)
    ≡ renameb Sb (substb fb' t)
substb-exts-rename-Sb-commute fb' t =
  beginEq
    substb (extsb fb') (renameb Sb t)
  ≡⟨ subst-renameb-comp Sb (extsb fb') t ⟩
    substb (λ x → renameb Sb (fb' x)) t
  ≡⟨ substb-rename-commute Sb fb' t ⟩
    renameb Sb (substb fb' t)
  □eq
```

Lemma 5.20. Compose of renaming and substitution:

```
-- substb and rename distribute over composition:
-- Renaming first with fb' and substituting with fb''
-- is the same as substituting with fb'' ∘ fb'
subst-renameb-comp : ∀ {Γ1 Γ2 Γ3 A}
  → (fb' : ∀ {Ab} → Ab ∈b Γ1 → Ab ∈b Γ2)
  → (fb" : ∀ {Ab} → □ Ab ∈b Γ2 → Γ3 ⊢b Ab)
  → (t : Γ1 ⊢b A)
  → substb fb" (renameb fb' t) ≡ substb (λ x → fb" (fb' x)) t
subst-renameb-comp fb' fb" (ε x) = refl
subst-renameb-comp fb' fb" (box t) = cong box (subst-renameb-comp fb' fb" t)
subst-renameb-comp fb' fb" (↚b t) =
  beginEq
    ↚b (substb (extsb fb") (renameb (extb fb') t))
  ≡⟨ cong ↚b (subst-renameb-comp (extb fb') (extsb fb") t) ⟩
    ↚b (substb (λ x → (extsb fb") ((extb fb') x)) t)
  ≡⟨ cong ↚b (substb-cong (λ x → extsb-comp fb' fb" x) t) ⟩
    ↚b (substb (extsb (λ x → fb" (fb' x))) t)
  □eq
```

Lemma 5.21. Congruence of substitution:

```
-- substb preserves pointwise equality
-- of renaming functions
substb-cong : ∀ {Γ₁ Γ₂ A}
  → {fb' : ∀ {Ab} → □ Ab ∈b Γ₁ → Γ₂ ⊢b Ab}
  → {fb" : ∀ {Ab} → □ Ab ∈b Γ₁ → Γ₂ ⊢b Ab}
  → (h : ∀ {Ab} → (x : □ Ab ∈b Γ₁) → fb' x ≡ fb" x)
  → (t : Γ₁ ⊢b A)
  → substb fb' t ≡ substb fb" t
substb-cong h (ε x) = h x
substb-cong h (box t) = cong box (substb-cong h t)
substb-cong h (ƛb t) = cong ƛb (substb-cong (extsb-cong h) t)
substb-cong h (t ·b t₁) = cong₂ _·b_ (substb-cong h t) (substb-cong h t₁)
```

5.3 Overview of the Formalisation

wat is er en hoe verhoudt het zich tot de scriptie qua resultaten

Chapter 6

Related Work

In this chapter you demonstrate that you are sufficiently aware of the state-of-art knowledge of the problem domain that you have investigated as well as demonstrating that you have found a *new* solution / approach / method.

Chapter 7

Conclusions

In this chapter you present all conclusions that can be drawn from the preceding chapters. It should not introduce new experiments, theories, investigations, etc.: these should have been written down earlier in the thesis. Therefore, conclusions can be brief and to the point.

Bibliography

[1] Thorsten Altenkirch and Bernhard Reus. Monadic presentations of lambda terms using generalized inductive types. In Jörg Flum and Mario Rodriguez-Artalejo, editors, *Computer Science Logic*, pages 453–468, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.

[2] Nick Benton, Chung-Kil Hur, Andrew J. Kennedy, and Conor McBride. Strongly typed term representations in coq. *Journal of Automated Reasoning*, 49(2):141–159, 2012.

[3] Alonzo Church. A formulation of the simple theory of types. *The Journal of Symbolic Logic*, 5(2):56–68, 1940.

[4] N.G de Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the church-rosser theorem. *Indagationes Mathematicae (Proceedings)*, 75(5):381–392, 1972.

[5] Jean-Yves Girard. Linear logic. *Theoretical Computer Science*, 50(1):1–101, 1987.

[6] José Espírito Santo, Luís Pinto, and Tarmo Uustalu. Modal Embeddings and Calling Paradigms. In Herman Geuvers, editor, *4th International Conference on Formal Structures for Computation and Deduction (FSCD 2019)*, volume 131 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 18:1–18:20, Dagstuhl, Germany, 2019. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik.

[7] A. S. Troelstra. Introductory note to 1933f. In Solomon Feferman, John W. Dawson Jr., Stephen C. Kleene, Gregory H. Moore, Robert M. Solovay, and Jean van Heijenoort, editors, *Collected Works, Volume I: Publications 1929–1936*, pages 296–299. Oxford University Press, New York, 1986.

[8] Philip Wadler, Wen Kokke, and Jeremy G. Siek. *De Bruijn Indices*. August 2022. Version 22.08, accessed 2025-06-06.

Appendix A

Appendix

Appendices are *optional* chapters in which you cover additional material that is required to support your hypothesis, experiments, measurements, conclusions, etc. that would otherwise clutter the presentation of your research.